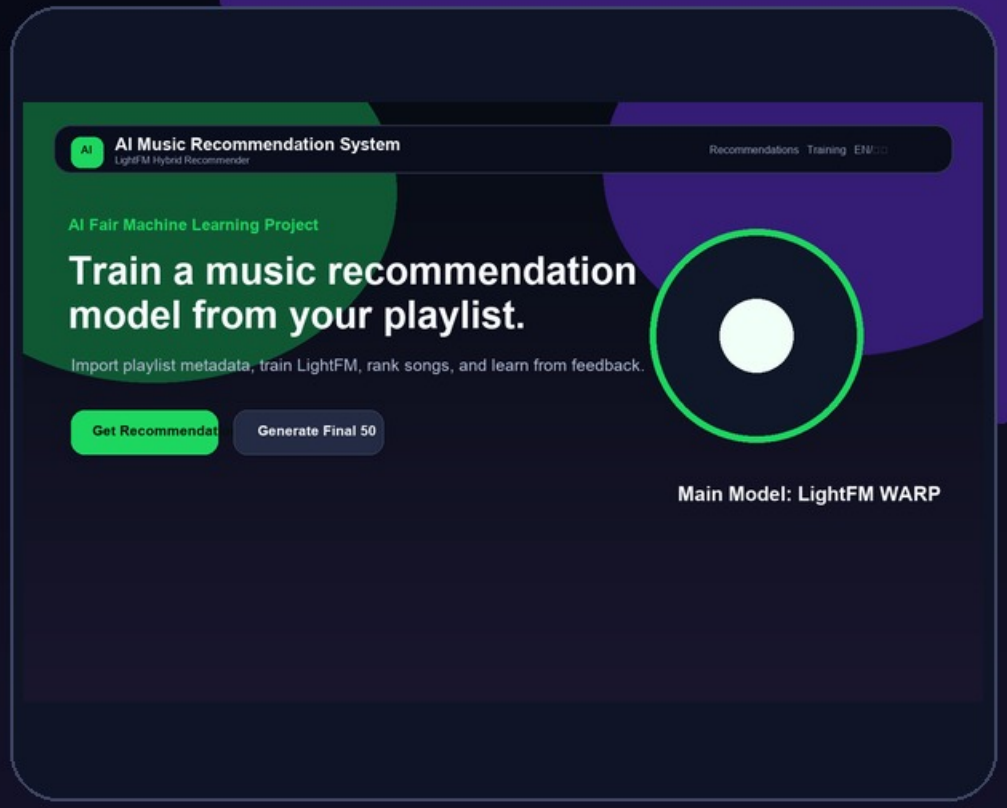
AI FAIR 2026 - MACHINE LEARNING TRACK

AI Music Recommendation System

Training a hybrid recommendation model from playlists, song features, and user feedback.



Main Model	Output	Storage
LightFM	Daily 50	CSV

Presenter: AI Fair Project Team

PROJECT OVERVIEW

The project solves a real music recommendation problem.

It learns from playlist behavior and feedback instead of returning random songs.

Problem

Music libraries are too large to browse manually. A recommender should learn the listener's taste.

Users

Students, music listeners, and AI Fair visitors who want to see a model learn from data.

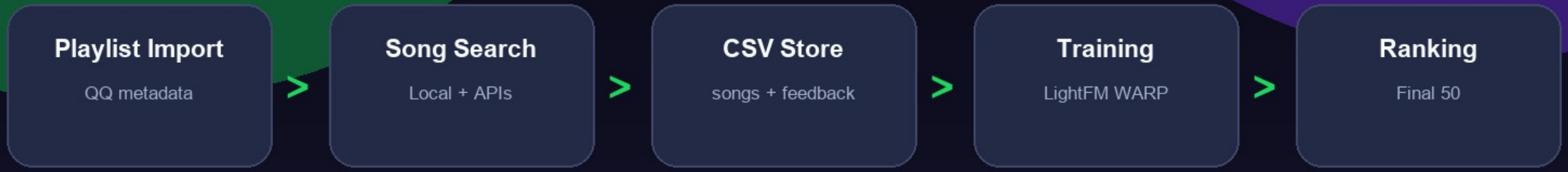
Goal

Show the complete machine learning loop: data, features, training, prediction, and feedback.

Thesis: this is not just a search page. It creates training data, trains a hybrid model, and updates recommendations after feedback.

SYSTEM ARCHITECTURE

A local Flask app turns music metadata and user actions into training data.



All data is stored locally. Online services are used only for public song metadata. The system does not download music audio.

DATASET AND FEATURES

The dataset is inspectable, structured, and ready for machine learning.

songs.csv	Song metadata + audio features	track, artist, genre, energy, tempo, popularity
interactions.csv	Training signals	playlist_import, manual_add, like, dislike
feedback.csv	Explicit feedback	Like = 1, Dislike = 0
playlists.csv	Import history	source, song_count, created_at

PLAYLIST IMPORT

QQ Music links are parsed safely, then metadata is imported.

- Supports playlist/{id}, taoge.html?id=, disstid=, tid=, and pure numeric IDs.
- Tries multiple metadata endpoints: qzone, v8 playlist, and musicu.fcg.
- Imports metadata only. It does not download music or bypass copyright.
- If QQ fails, the user can use manual input or song search.

URL extraction example

QQ playlist URL
> extract playlist_id
> fetch metadata
> normalize songs
> save songs.csv
> create interactions

MUSIC SEARCH APIS

The app searches local data first, then public music metadata APIs.

1 **Local CSV**
Fast, stable, includes model-ready features.

2 **Deezer**
Public metadata for mainstream songs.

3 **MusicBrainz**
Open music database, rate limited safely.

4 **Last.fm**
Optional API key. Skipped if missing.

Every request uses timeout=8 and try/except, so API failure does not crash the Flask app.

FEATURE ENGINEERING

Songs become item feature tokens that LightFM can learn from.

genre:pop

artist:the_weeknd

tag:dance

source:qq

energy:high

tempo:fast

valence:positive

danceability:high

popularity:high

The model learns from interpretable features such as high energy, fast tempo, genre, artist, source, and mood.

TRAINING DATA

Playlist actions and feedback become a user-song interaction matrix.

playlist_import	manual_add	like	dislike
+1.0	+1.0	+2.0	-1.0

LightFM WARP trains from positive ranking examples. Dislikes are stored and used during re-ranking to lower scores.

MODEL AND METHOD

LightFM is the main machine learning model because it supports hybrid recommendation.

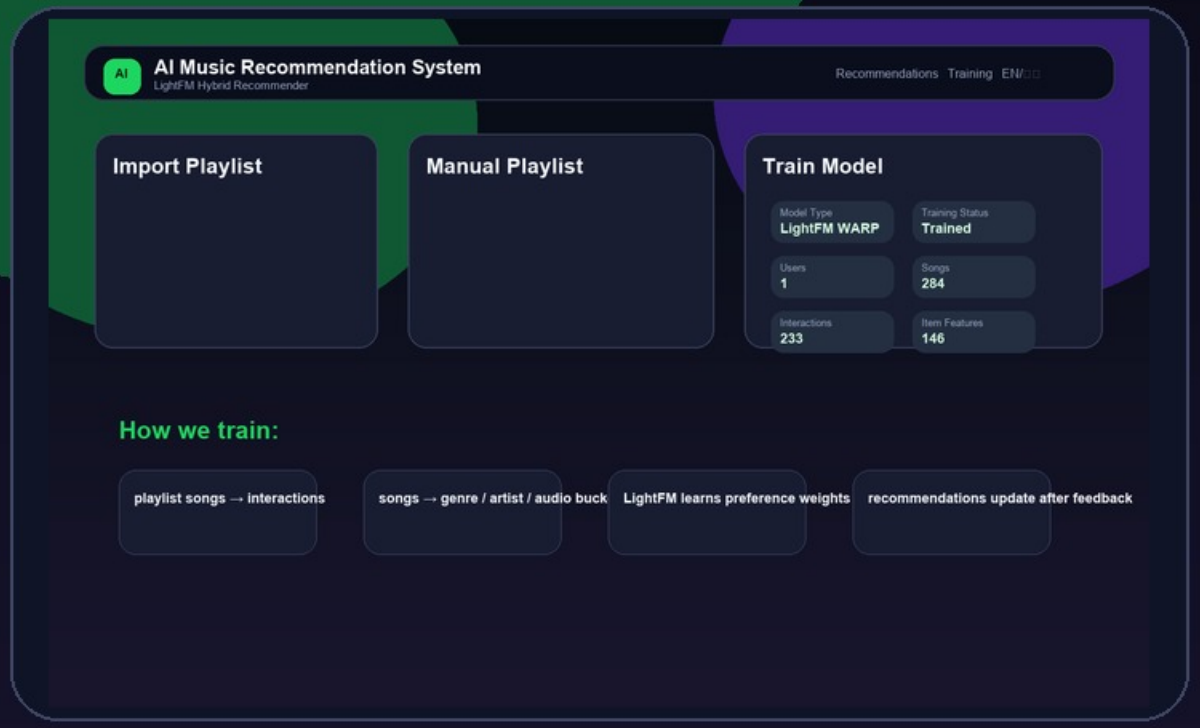
- Open-source recommender library.
- Learns from implicit and explicit feedback.
- Uses WARP ranking loss for Top-N recommendation.
- Combines interactions with item features.
- Works well for a school AI Fair dataset.

Training object

```
model = LightFM(loss="warp")
model.fit(interactions,
item_features,
sample_weight,
epochs=30)
```

TRAINING WORKFLOW

The UI exposes the machine learning pipeline.



- Import or manually add songs.
- Write interactions.csv rows.
- Build item feature tokens.
- Train LightFM WARP.
- Save lightfm_model.pkl.
- Show model status in the UI.

RECOMMENDATION RANKING

Final recommendations blend learned preference and content similarity.

final_score = 0.7 x model_score + 0.3 x content_similarity_score

model_score

The trained LightFM model predicts how much the user may like each candidate song.

content_similarity

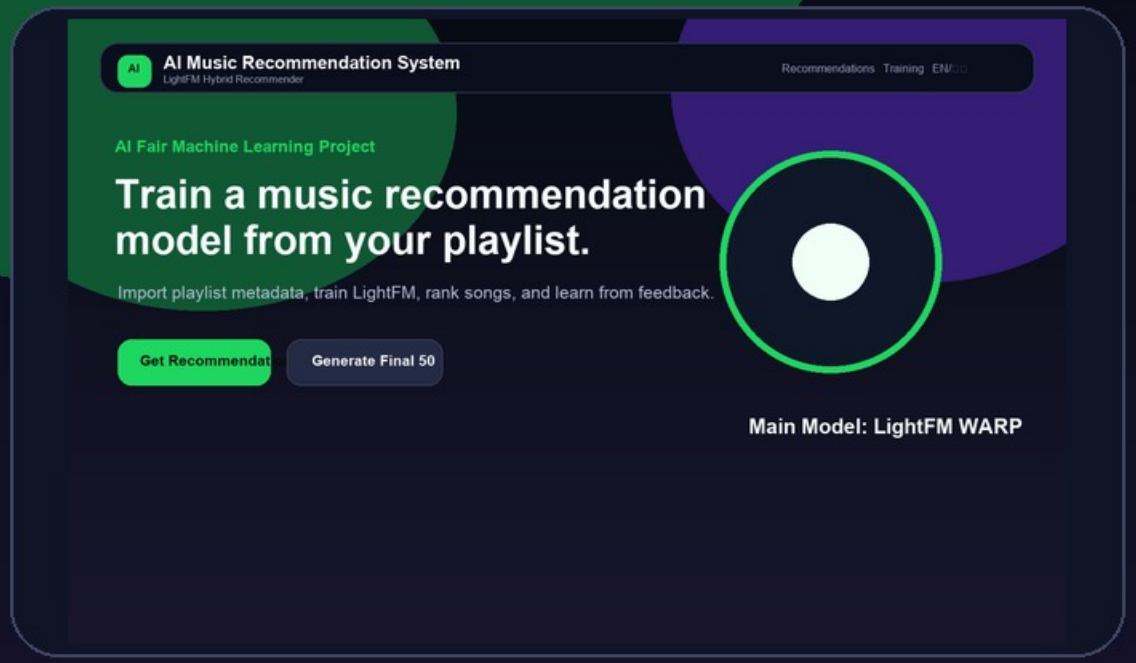
Cosine similarity compares audio features such as energy, tempo, danceability, and mood.

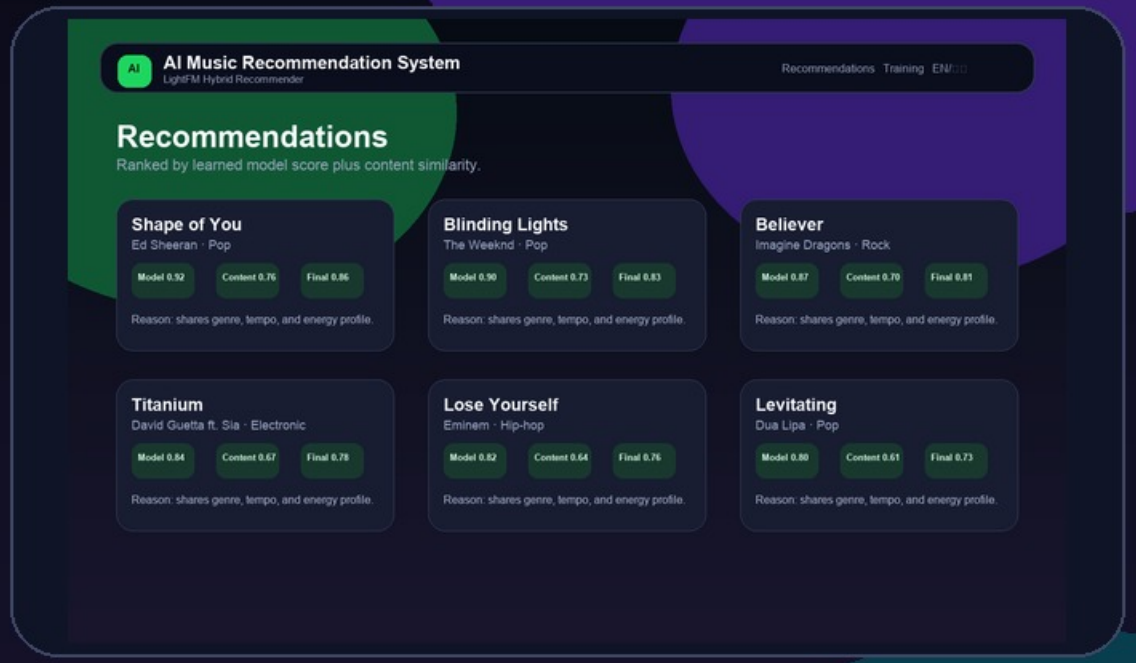
rule explanation

Reasons are generated deterministically, not by a language model.

SOFTWARE EXPERIENCE

The app looks and feels like a modern music recommendation dashboard.

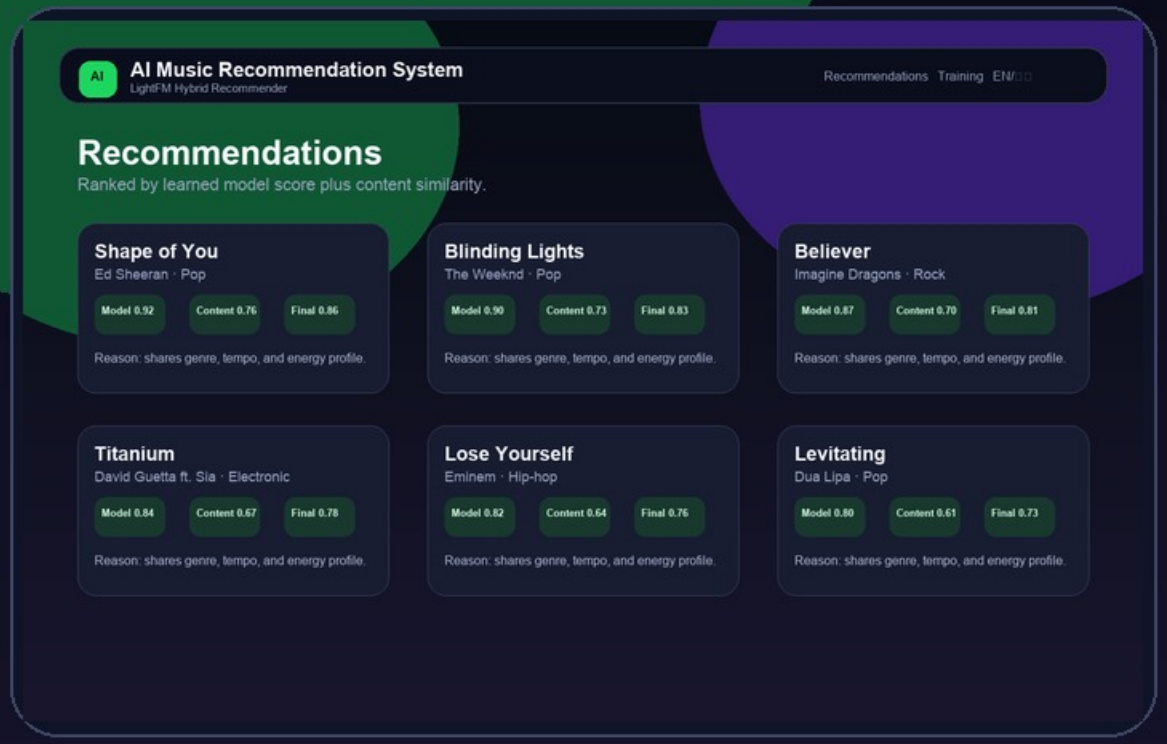




Bilingual UI, playlist import, manual songs, model status, recommendation cards, Like/Dislike, and Daily 50 output.

RESULTS AND EVALUATION

The system produces ranked songs, explanations, and a final 50-song daily mix.



Top Cards
10

Final Mix
50

Explanation
Rules

Feedback
Retrain

After Like/Dislike feedback, interactions.csv changes, the model retrains, and future rankings shift.

FINAL OUTPUT

Daily Recommendation Mix turns model scores into a show-ready playlist.

- Generates up to 50 songs.
- Filters songs already used for training.
- Reduces repeated artists and genres.
- Can be copied as playlist text output.

REAL-WORLD USE

The same pipeline can support personal and school music discovery.

For students

Learn ML from a product everyone understands: playlists and song recommendations.

For music apps

Convert playlist behavior and feedback into a recommendation loop.

Future work

Add richer audio analysis, multi-user data, metrics, and stronger diversity controls.

Most useful improvement: collect more feedback over time and evaluate precision@k and recall@k.

ETHICAL AI DISCLOSURE

The project uses AI help responsibly and avoids copyright risks.

- AI assistance was used for coding, debugging, UI writing, and documentation support.
- The model and code were checked and understood by the student.
- No local LLM, LM Studio, Ollama, or OpenAI API is used for explanations.
- QQ import uses metadata only, with no audio download or unauthorized playback.

Risk controls

Timeouts for API requests, friendly error fallback, local CSV storage, rule-based explanations, and no private audio scraping.

LIMITATIONS

The model is real, but dataset size and public metadata sources limit accuracy.

Small dataset

A single demo user is useful for demonstration, not production-scale personalization.

Estimated features

Some online results do not provide true audio features, so the app estimates from metadata.

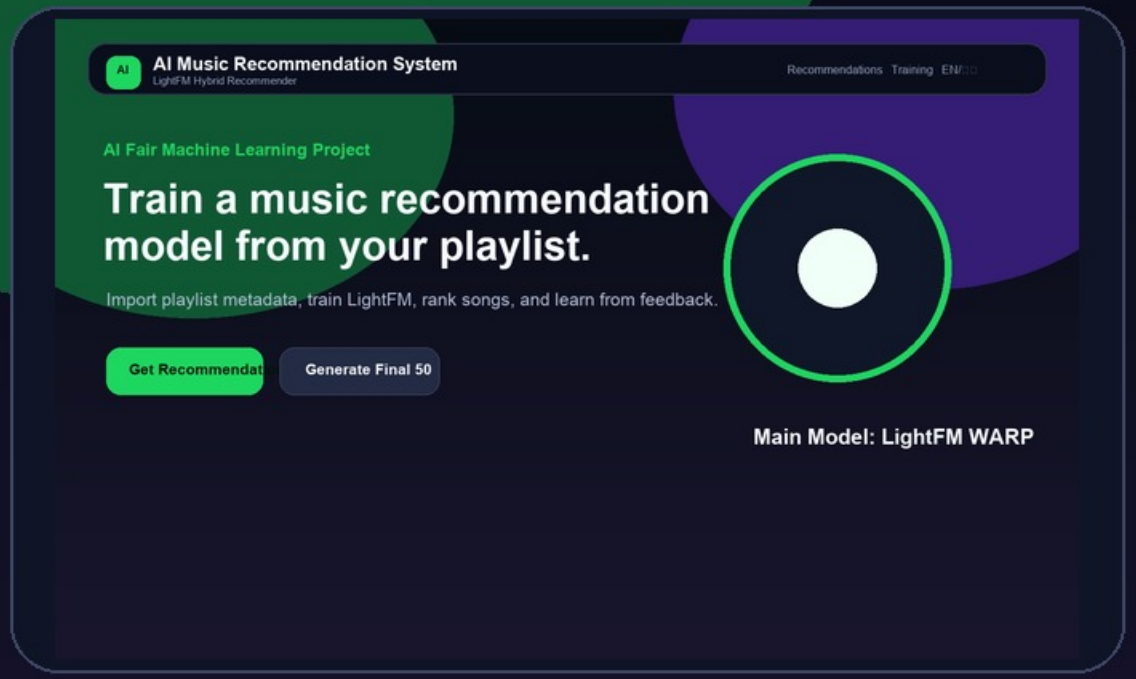
Unstable APIs

QQ Music public endpoints may change, so the app includes fallback and manual input.

Future improvements: real audio-feature sources, multi-user data, train/test evaluation, precision@k, recall@k, and stronger diversity re-ranking.

HOW TO USE

Quick start: open the app, import music taste data, then train the model.



1. **Start the app**
   Run python app.py and open the local URL.
2. **Import or add songs**
   Paste a QQ playlist link or type Song - Artist lines.
3. **Check the dataset**
   The app saves local CSV training data.
4. **Train the model**
   Click Train Model to build LightFM.

DEMO TUTORIAL

A live demo can show the full ML loop in under three minutes.

Step 1 — **Search or import**
Use familiar songs or a QQ playlist.

Step 2 — **Create interactions**
Add songs to the training playlist.

Step 3 — **Train LightFM**
Show users, songs, interactions, and item features.

Step 4 — **Recommend**
Show Top 10 scores and rule-based reasons.

Step 5 — **Feedback**
Click Like or Dislike to update training data.

Step 6 — **Final 50**
Generate a Daily Recommendation Mix.

Key line: every click creates data that can change the trained model.

USAGE NOTES

If an online service fails, the system still works through local data and manual input.

If QQ import fails

Use manual playlist input or Search Songs. Public QQ endpoints can change or block some playlists.

If API search is slow

The app uses timeouts and skips failed providers so Flask stays running.

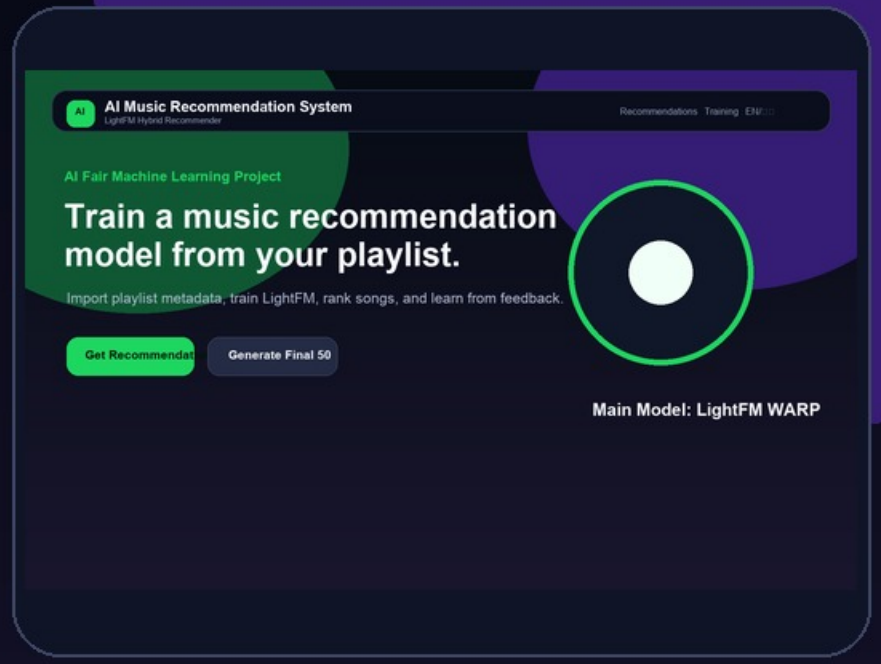
If LightFM is unavailable

The app falls back to content similarity and a logistic regression feedback model.

The project is designed for live demos: API errors become friendly messages while local data and the recommender still work.

CLOSING

This project shows the full ML loop: data, features, training, ranking, and feedback.



Playlist import creates implicit feedback. Like/Dislike creates explicit feedback. LightFM learns preferences and ranks songs the user is likely to enjoy.

Thank you. Questions?